TrialBridge — Design Document

Project Title: TrialBridge: AI-Powered Clinical Trial Patient Matching Platform

---

1. System Overview

TrialBridge is a full-stack platform designed to automate the process of matching patients to clinical trials using AI. At its core, it is a FastAPI backend that exposes REST endpoints for trial search, patient matching, and eligibility scoring. The backend talks to a PostgreSQL database that holds both clinical trial records (scraped from ClinicalTrials.gov) and patient records (imported from Kaggle and FHIR datasets). The intelligence layer consists of a pipeline of LLM-based agents that handle everything from understanding natural language criteria to generating database queries to computing multi-factor eligibility scores.

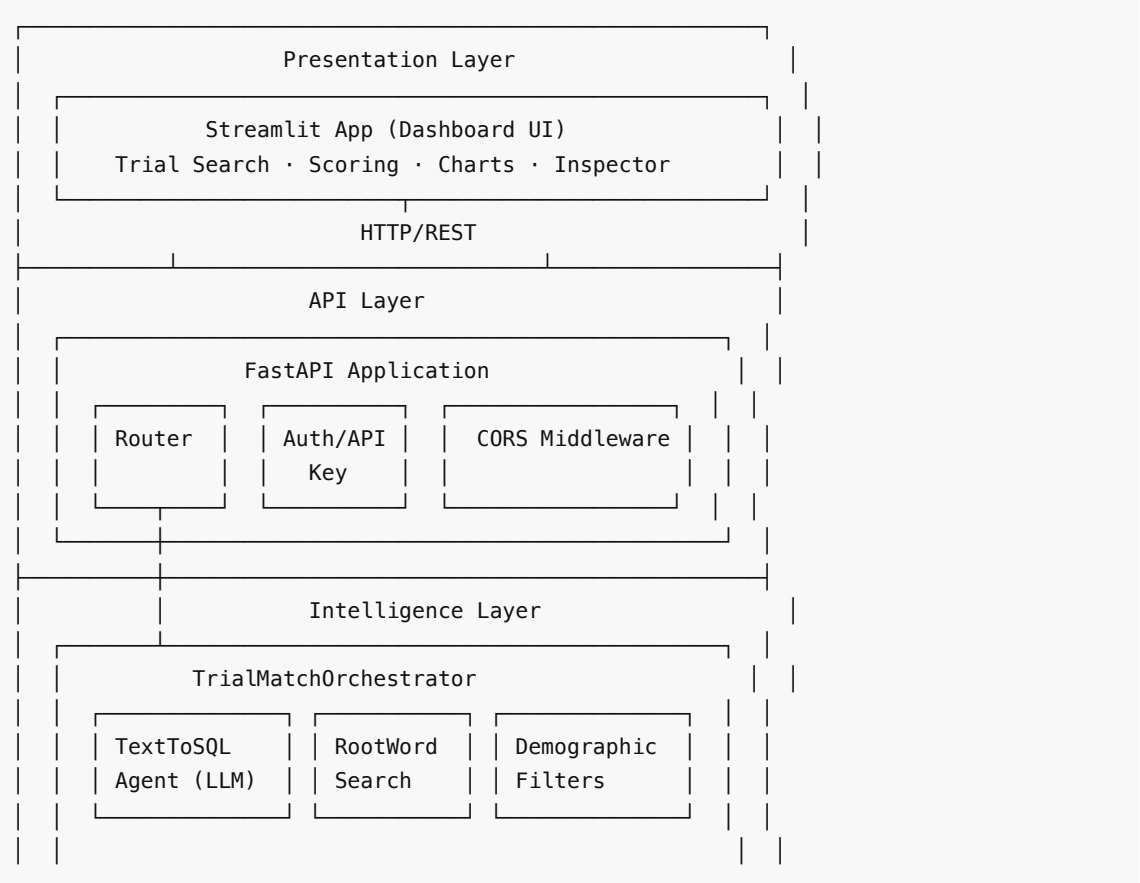
The user-facing layer is a Streamlit analytics dashboard that provides interactive visualizations and patient inspection tools, communicating with the backend through the REST API.

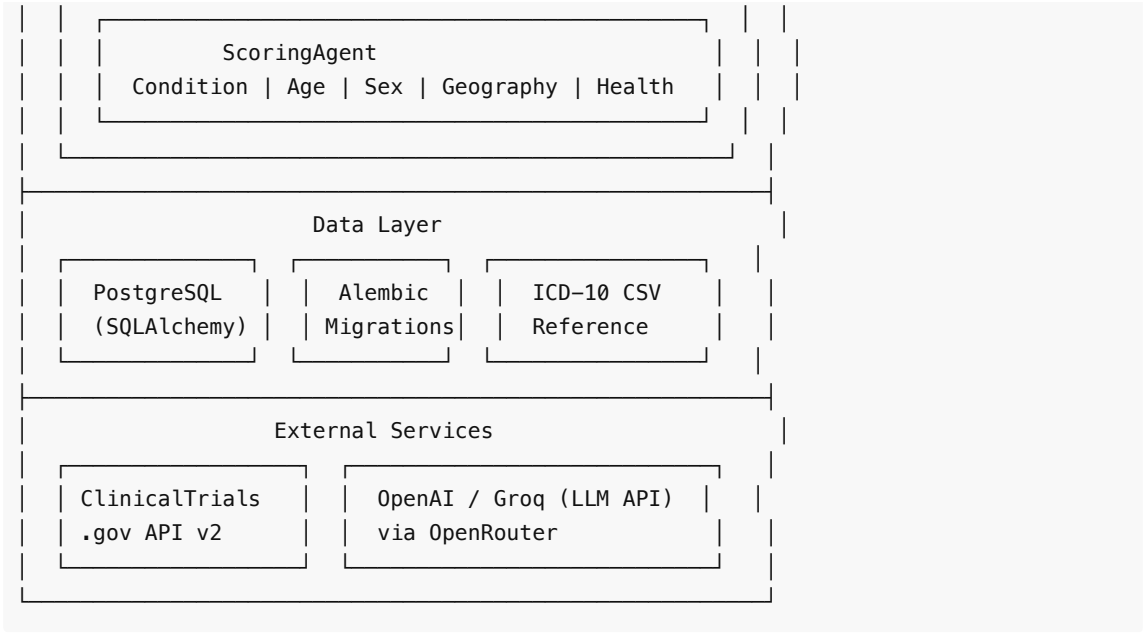
The whole system is containerized with Docker for reproducible deployment and uses Alembic for database schema migrations, ensuring the database evolves safely alongside the application code.

2. Architecture

2.1 High-Level Architecture

The system follows a layered architecture pattern where each layer has a clear responsibility and communicates only with its immediate neighbors:





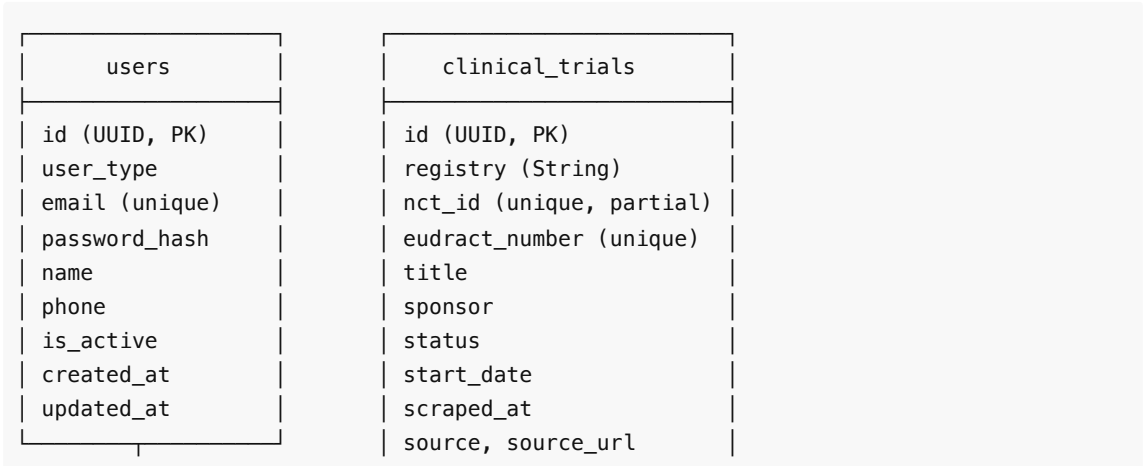
2.2 Component Interaction Flow

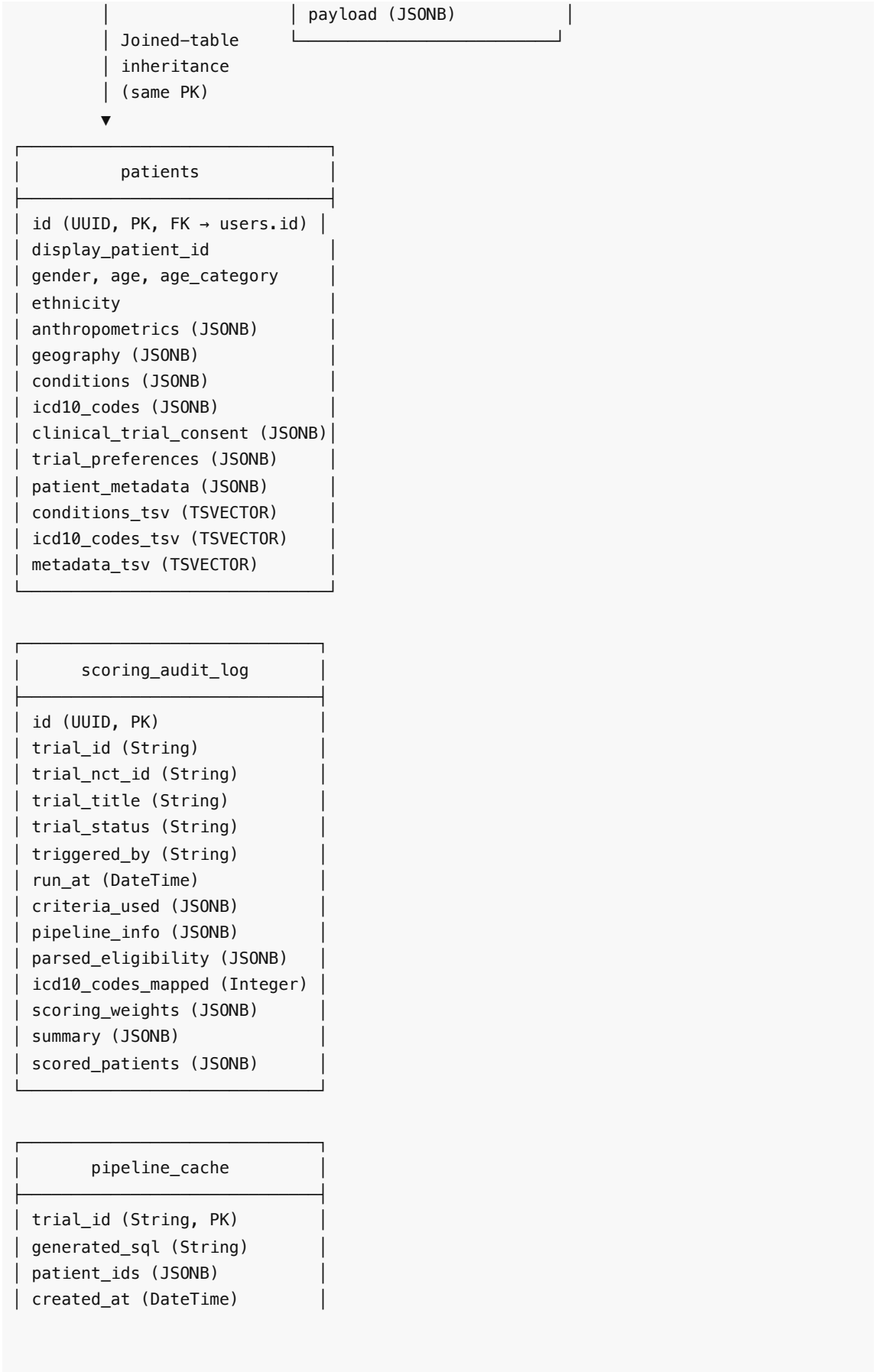
When a user triggers a scoring request, the flow moves through the system like this:

1. The Streamlit dashboard sends an HTTP POST to the FastAPI backend.
2. The API route handler validates the request using Pydantic schemas and obtains a database session.
3. The ScoringAgent is instantiated, which internally creates a TrialMatchOrchestrator instance.
4. The orchestrator fetches trial details from PostgreSQL, then runs the three-stage matching pipeline (Text-to-SQL → Root Word Search → Demographic Filters).
5. Matched patients are passed to the scoring engine, which computes composite scores using ICD-10 mapping and weighted dimension scoring.
6. Results are assembled into a structured response with summary statistics, patient breakdowns, and an audit log entry.
7. The response flows back through the API layer to the frontend, which renders the visualizations.

3. Database Design

3.1 Entity Relationship Diagram




payload (JSONB)
Joined-table
inheritance
(same PK)
patients
id (UUID, PK, FK → users.id)
display_patient_id
gender, age, age_category
ethnicity
anthropometrics (JSONB)
geography (JSONB)
conditions (JSONB)
icd10_codes (JSONB)
clinical_trial_consent (JSONB)
trial_preferences (JSONB)
patient_metadata (JSONB)
conditions_tsv (TSVECTOR)
icd10_codes_tsv (TSVECTOR)
metadata_tsv (TSVECTOR)
scoring_audit_log
id (UUID, PK)
trial_id (String)
trial_nct_id (String)
trial_title (String)
trial_status (String)
triggered_by (String)
run_at (DateTime)
criteria_used (JSONB)
pipeline_info (JSONB)
parsed_eligibility (JSONB)
icd10_codes_mapped (Integer)
scoring_weights (JSONB)
summary (JSONB)
scored_patients (JSONB)
pipeline_cache
trial_id (String, PK)
generated_sql (String)
patient_ids (JSONB)
created_at (DateTime)

```
| expires_at (DateTime)          |
└────────────────────────────────┘
```

3.2 Design Decisions

Why JSONB for trial payloads? Clinical trial data from ClinicalTrials.gov has a deeply nested, semi-structured format that varies between studies. Some trials have detailed location data with GPS coordinates, others have just a city name. Some list 15 conditions, others list one. Trying to normalize all of this into rigid relational columns would mean constantly altering the schema as we encounter new data shapes. Instead, we store the raw API response as a JSONB column and extract the fields we query frequently (nct_id, title, status) into indexed columns. This gives us the best of both worlds — fast lookups on common fields, and full flexibility for everything else.

Why tsvector on patient conditions? PostgreSQL's full-text search with tsvector handles stemming natively. When we index the patient's conditions text as a tsvector, a search for "diabetic" automatically matches "diabetes," "diabetics," and "diabetic." This is critical for medical terminology where the same condition can be expressed in many grammatical forms. The tsvector column is pre-computed and stored, so searches are fast even across 21,000+ patient records.

Why UUID primary keys? The system is designed to eventually support distributed deployments where data might be synchronized across multiple instances. Auto-incrementing integer IDs create conflicts in distributed scenarios. UUIDs are globally unique by definition, making future scaling straightforward.

Why a separate users table linked to patients? Not every user is a patient. The system supports different user types (patients, researchers, administrators). By separating user authentication concerns (email, password, user type) from patient medical data (conditions, vitals, demographics), we maintain clean separation of concerns and can add new user types without touching the patient schema.

4. API Design

4.1 Endpoint Overview

All endpoints are versioned under `/api/v1` and protected by API key authentication via the `X-API-Key` header.

Method	Endpoint	Purpose
GET	`/api/v1/agents/search-trials`	Search and browse clinical trials
POST	`/api/v1/agents/match-patients`	AI-powered patient-to-trial matching
POST	`/api/v1/agents/match-patients-criteria`	Match patients using natural language criteria
POST	`/api/v1/agents/score-patients`	Multi-factor eligibility scoring for a trial
POST	`/api/v1/agents/score-patients-criteria`	Score patients using free-text criteria
POST	`/api/v1/agents/text-to-sql`	Generate SQL from natural language
POST	`/api/v1/agents/root-word-search`	Search patients using stemmed keywords
POST	`/api/v1/agents/extract-root-words`	Extract root words from a search query

4.2 Request/Response Examples

Score Patients Request:

```
{
  "trial_id": "NCT06230718",
  "limit": 200,
  "source": "kaggle"
}
```

Score Patients Response (abbreviated):

```
{
  "trial_nct_id": "NCT06230718",
  "trial_title": "A Study of Tirzepatide in Participants With Type 2 Diabetes",
  "trial_status": "RECRUITING",
  "summary": {
    "total_scored": 187,
    "eligible_count": 142,
    "excluded_count": 45,
    "avg_score": 62.4,
    "max_score": 89.0,
    "min_score": 18.5
  },
  "scored_patients": [
    {
      "patient_id": "a1b2c3d4-...",
      "composite_score": 89.0,
      "excluded": false,
      "breakdown": {
        "condition": { "score": 28.5, "max": 30, "details": "..." },
        "age": { "score": 18.0, "max": 20, "details": "..." },
        "sex": { "score": 15.0, "max": 15, "details": "..." },
        "geography": { "score": 12.5, "max": 15, "details": "..." },
        "health": { "score": 15.0, "max": 20, "details": "..." }
      },
      "patient_summary": {
        "age": 58,
        "gender": "Female",
        "state": "California",
        "conditions": ["Type 2 Diabetes", "Hypertension"],
        "bmi": 31.2,
        "smoker_status": "Never"
      }
    }
  ],
  "parsed_eligibility": {
    "inclusion_rules": ["Adults aged 18–75", "Diagnosed with T2D"],
    "exclusion_rules": ["Type 1 diabetes", "Pregnancy"],
    "excluded_conditions": ["Type 1 Diabetes"]
  },
```

```
  "audit_log_id": "e5f6g7h8-..."
}
```

4.3 Authentication

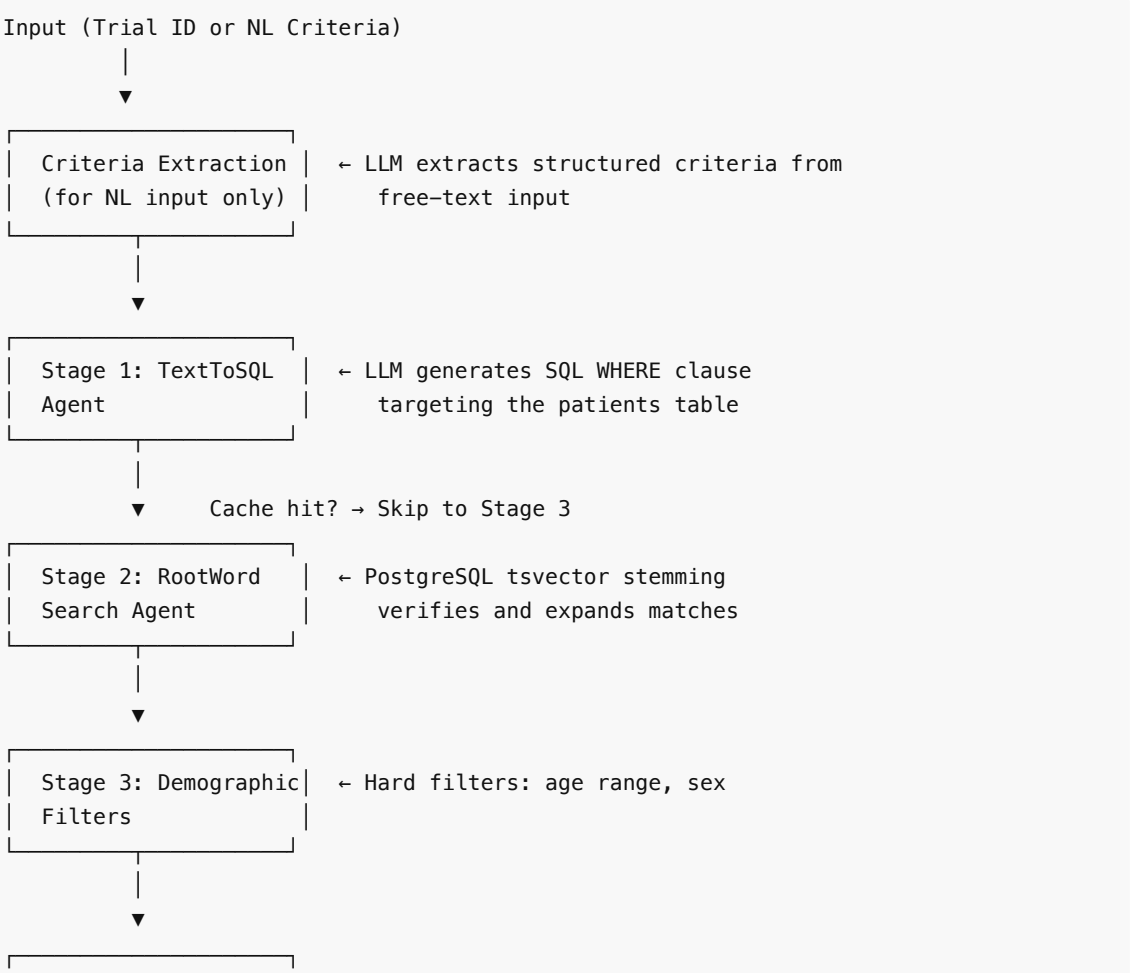
The API uses a simple API key mechanism for the current version. Every request must include an `X-API-Key` header with a value matching the `API_KEY` environment variable. The system also supports JWT-based authentication (with `SECRET_KEY` and configurable token expiry) for future user-level access control, though this is not enforced on the agent endpoints currently.

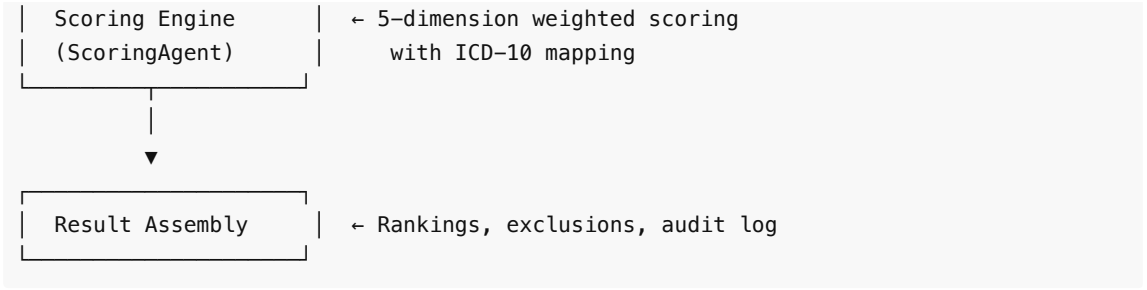
Rate limiting is implemented as a sliding-window counter — by default, 100 requests per 60 seconds per client, with a 60-second cooldown penalty if the limit is exceeded. This protects the LLM API calls from being overwhelmed by rapid-fire requests.

5. AI Agent Pipeline Design

5.1 Pipeline Architecture

The intelligence layer is built as a chain of specialized agents, each handling one aspect of the matching problem. This design was chosen over a single monolithic LLM call for several reasons — it is easier to debug (you can see exactly which stage failed), easier to cache (Stage 1 results can be reused across multiple scoring runs for the same trial), and more resilient (if the LLM fails, the root-word search agent provides a fallback).





5.2 TextToSQL Agent

This agent receives the trial's conditions and a description of the patient table schema, and asks the LLM to generate a SQL WHERE clause that would select matching patients. The prompt is carefully engineered to:

- Include the exact column names and types from the patients table
- Specify that the tsvector column should be used for text matching
- Request ILIKE patterns for condition matching
- Ask for conservative matching (better to include too many than miss good candidates)

The LLM response is parsed to extract just the WHERE clause, which is then parameterized and executed against the database. If the LLM returns malformed SQL or the query returns zero results, the system falls back to Stage 2.

Results from this stage are cached in a DbPipelineCache with a 5-minute TTL, keyed by trial NCT ID. This means if multiple researchers score the same trial within a 5-minute window, only the first request triggers an LLM call.

5.3 Root Word Search Agent

This agent uses a fundamentally different approach from the TextToSQL agent. Instead of asking an LLM to generate SQL, it programmatically extracts root words from the trial's conditions and uses PostgreSQL's `to_tsquery` function to search the pre-indexed tsvector column on the patients table.

The advantage is that this approach is deterministic — given the same input, it always produces the same results. It also handles morphological variations naturally (the stemmer reduces "hypertensive" and "hypertension" to the same root). The downside is that it cannot understand semantic relationships (it would not know that "heart attack" and "myocardial infarction" are the same thing), which is where the LLM-based Stage 1 adds value.

5.4 Scoring Engine

The ScoringAgent computes a composite score for each patient across five dimensions:

Condition Match (30 points): This is the most complex dimension. The agent maintains a mapping of common medical conditions to ICD-10 code prefixes (for example, "Type 2 Diabetes" maps to E11, "Hypertension" maps to I10-I15). It also loads the full ICD-10 reference CSV (14MB, ~70,000 codes) to handle less common conditions. For each patient, it checks how many of their conditions overlap with the trial's target conditions, using both direct string matching and ICD-10 code matching. The score is proportional to the overlap percentage, with bonuses for exact matches on primary conditions.

Age Fit (20 points): The trial specifies a minimum and maximum age. Patients at the center of this range get full marks. The score decreases linearly as the patient's age approaches the edges of the range. Patients outside the range entirely get zero and are flagged for exclusion.

Sex Alignment (15 points): If the trial specifies a sex requirement (for example, a breast cancer trial might require female participants), this is a binary check — match gets full marks, mismatch gets zero and triggers exclusion. If the trial accepts all sexes, every patient gets full marks.

Geographic Proximity (15 points): The agent checks if the patient's state matches any of the trial's site locations. An exact state match gets full marks. Being in the same country but a different state gets partial credit. Being in a different country gets zero.

Health Status (20 points): This dimension uses the patient's self-reported general health status (excellent, very good, good, fair, poor), BMI, and smoking status. Each factor is mapped to a numerical score, and the dimension total is a weighted combination. Patients with "excellent" health get full marks on the health status component, while "poor" health gets minimal marks. BMI within the normal range (18.5-25) gets a bonus; extreme values get penalties. Non-smokers get a small bonus over current smokers.

6. Frontend Design

6.1 Streamlit Dashboard

The dashboard is built as a single-page Streamlit application with a dark theme and custom typography (Figtree for body text, Overpass Mono for numerical values). The design philosophy was to make it feel like a professional analytics tool rather than a typical Streamlit prototype.

Layout Structure:

- **Header:** Project title and subtitle ("Patient screening for clinical trials")
- **Input Panel:** Radio buttons for data source (Kaggle vs FHIR) and input mode (Select Trial vs Custom Input). In trial selection mode, a dropdown shows available trials with NCT ID and conditions. In custom mode, a text area accepts free-text criteria. A slider controls the maximum number of patients to score.
- **Results Section:** Appears after clicking "Run Scoring"
  - Summary stat bar showing total scored, eligible, excluded, average/high/low scores, and ICD-10 codes mapped
  - Criteria panel showing how the system interpreted the input
  - Expandable eligibility rules panel with inclusion/exclusion criteria
  - Score distribution histogram (Plotly)
  - Top patient radar chart showing the 5-dimension breakdown
  - Ranked patient table with tabs for eligible, excluded, and raw JSON
  - Patient inspector with demographic detail grid and score bars

7. Data Pipeline Design

7.1 Clinical Trial Scraper

The scraper targets the ClinicalTrials.gov API v2 endpoint (`https://clinicaltrials.gov/api/v2/studies`) and fetches studies with the following filters:

- Status: RECRUITING
- Country: United States
- Study type: INTERVENTIONAL

It processes results in paginated batches, extracting and normalizing the following fields for each trial:

- NCT ID, title, and overall status
- Conditions being studied

- Eligibility criteria text (inclusion and exclusion)
- Age and sex eligibility
- Study phase
- Sponsor and collaborator information
- Site locations (facility name, city, state, zip, country)
- Keywords and study design details

The raw API response is stored as a JSONB payload alongside the extracted fields, so nothing is lost even if we do not explicitly parse every field.

The scraper uses an HTTP session with retry logic (5 retries with exponential backoff) and handles API rate limiting gracefully. It performs upserts based on NCT ID, so running it multiple times updates existing records rather than creating duplicates.

7.2 Patient Data Importer

The Kaggle patient importer uses a producer-consumer pattern with an asyncio queue:

- A single producer thread reads the CSV file line by line (streaming, not loading the entire file into memory)
- Multiple consumer workers (configurable, default 4) pull records from the queue
- Each worker maintains its own async database session
- Records are committed in configurable batch sizes (default 50)
- A progress tracker logs throughput statistics every 500 records

This architecture allows the system to import 21,000+ patient records in under 5 minutes on a standard development machine, while keeping memory usage constant regardless of dataset size.

Each patient record is linked to a synthetic user account (auto-generated email and password hash) because the system's data model expects every patient to have an associated user. The importer maps CSV columns to the patient schema, including deriving fields like BMI category and age from the raw data.

8. Security Design

8.1 API Authentication

- **API Key:** All agent endpoints require a valid `X-API-Key` header. The key is set as an environment variable (`API_KEY` in `.env`) and validated on every request through a FastAPI dependency using constant-time comparison (`compare_digest`). Rate limiting runs before API key verification so abusive clients are throttled even before the key check.
- **Password Hashing:** User passwords are hashed using bcrypt with automatic salt generation during patient data import.

8.2 Rate Limiting

The rate limiter uses a sliding-window algorithm implemented as an in-memory service:

- Tracks request timestamps per client (identified by API key or IP)
- Allows a configurable number of requests within a time window (default: 100 requests per 60 seconds)
- Applies a cooldown penalty when the limit is exceeded (default: 60 seconds)
- Rate limit parameters are configurable via environment variables

8.3 CORS Configuration

CORS is configured through the `ALLOWED_ORIGINS` environment variable, which accepts a comma-separated list of allowed origins. In development, this is typically set to `http://localhost:8501` (Streamlit dashboard).

8.4 Environment Variable Management

Sensitive configuration (database credentials, API keys, secret keys) is managed through `.env` files that are excluded from version control via `.gitignore`. The application validates all required environment variables at startup using Pydantic's settings validation — if a required variable is missing, the application fails fast with a clear error message rather than crashing later with a cryptic error.

9. Deployment Architecture

9.1 Docker Configuration

The application uses a multi-stage Docker build for production:

- **Stage 1 (build):** Python 3.13.9-slim base with shared environment variables
- **Stage 2 (dependencies):** Installs build-essential and compiles Python wheels, runs `uv sync --frozen` to install exact pinned dependencies from the lockfile
- **Stage 3 (production):** Copies the virtual environment and application code, exposes port 8000, runs uvicorn

The Docker Compose configuration defines a single `api` service that builds from the Dockerfile and injects environment variables from the host's `.env` file.

9.2 Production Considerations

- **Gunicorn:** A gunicorn configuration file is included for running multiple uvicorn workers behind gunicorn in production, with configurable worker count and connection pooling
- **Database Pooling:** SQLAlchemy is configured with connection pool size (default 20) and max overflow (default 10) to handle concurrent requests efficiently
- **Logfire Integration:** In production mode, the application integrates with Logfire for structured observability, including automatic FastAPI instrumentation for request tracing
- **Frozen Dependencies:** Production deployments use `uv sync --frozen` to ensure the exact dependency versions from the lockfile are installed, preventing unexpected updates

10. Testing Strategy

10.1 Test Structure

Tests are organized under the `tests/` directory using pytest with async support:

- **conftest.py:** Session-scoped fixtures for database setup, async session factory, and test data seeding
- **test_models.py:** Unit tests for SQLAlchemy models — verifying field types, constraints, relationships, and default values

10.2 Test Configuration

- Tests use `pytest-asyncio` with `asyncio_mode = "auto"` for seamless async test execution
- Code coverage is measured with `pytest-cov`
- Code quality is enforced through `ruff` (linting) and `black` (formatting)
- Type checking is done with `mypy` in strict mode (`disallow_untyped_defs = true`)

11. Caching Strategy

11.1 Pipeline Cache

The DbPipelineCache provides **database-backed caching** for the matching pipeline's intermediate results. Unlike a simple in-memory dict, it is persisted in PostgreSQL (the `pipeline_cache` table), which means it survives process restarts and is shared across all Gunicorn/uvicorn workers without inter-process locking. When the TextToSQL agent generates a query for a specific trial, the resulting patient IDs and generated SQL are cached with a 5-minute TTL via upsert. If another scoring request for the same trial comes in within that window, the system skips the LLM call entirely and uses the cached patient set. Expired entries are lazily evicted on read and can be batch-evicted via `evict_expired()`.

This is particularly important because LLM API calls are the slowest part of the pipeline (typically 2-5 seconds per call) and the most expensive in terms of API costs. Caching eliminates redundant calls when multiple researchers are evaluating the same trial.

11.2 Valid Trials Cache

The list of valid trial IDs (trials confirmed to have complete data suitable for scoring) is stored as a JSON file (`data/valid_trial_ids.json`) and loaded once at application startup using Streamlit's `@st.cache_data` decorator. This avoids hitting the database on every page load just to populate the trial selector dropdown.

12. Logging and Observability

12.1 Structured Logging

The application uses Python's standard logging module with a custom configuration that supports two output modes:

- **Development:** Human-readable formatted logs with color coding and module/line information
- **Production:** JSON-structured logs compatible with log aggregation tools, with Logfire integration for distributed tracing

12.2 Audit Logging

Every scoring run creates a `ScoringAuditLog` entry in the database containing:

- The trial ID, NCT ID, title, and status
- Who triggered the run (`triggered_by`)
- The full criteria used (JSONB)
- Pipeline info — pre-filter counts, generated SQL, cache hit status, per-stage timings (JSONB)
- Parsed eligibility rules — inclusion, exclusion, excluded conditions (JSONB)
- Number of ICD-10 codes mapped
- Scoring weights used
- Summary statistics — total scored, eligible, excluded, avg/max/min scores (JSONB)
- The complete scored patients list with breakdowns (JSONB)
- Timestamp of the scoring run

This audit trail serves two purposes — debugging (if a scoring result looks wrong, you can trace back exactly what happened) and analytics (tracking which trials are being scored most frequently, how long the pipeline takes on average, and how LLM response times trend over time).

13. Future Enhancements

While the current version demonstrates the core concept successfully, several enhancements are planned for future iterations:

- **EHR Integration:** Connecting to live electronic health record systems (FHIR R4 APIs from Epic, Cerner) to score real patients in real time, rather than working with static datasets.
- **Multi-Trial Scoring:** Scoring a single patient against multiple trials simultaneously to identify all trials they might qualify for, rather than scoring many patients against one trial at a time.
- **Collaborative Filtering:** Using historical enrollment data to improve scoring — if patients with a certain profile have historically been successful in similar trials, boost their scores.
- **Patient Notifications:** A patient-facing interface where individuals can see which trials they might qualify for and express interest in participating.
- **Advanced NLP:** Fine-tuning a domain-specific language model on clinical trial criteria to improve extraction accuracy beyond what general-purpose LLMs achieve.
- **Geographic Intelligence:** Using actual distance calculations between patient addresses and trial sites rather than simple state-level matching.